

Министерство образования Республики Беларусь
Учреждение образования
“Брестский государственный технический университет”
Факультет электронно-информационных систем
Кафедра интеллектуальных информационных технологий

Отчёт по лабораторной работе № 7
По дисциплине «Современные платформы программирования»

Выполнил:
Тютюков К. О.
Студент группы ПО-13
Проверил:
А.А. Крощенко
Доцент кафедры ИИТ
03.04.2026

Цель работы: освоить возможности языка программирования Python в разработке оконных приложений

Ход работы:

Задание 1. Построение графических примитивов и надписей

Требования к выполнению

- Реализовать соответствующие классы, указанные в задании;
- Организовать ввод параметров для создания объектов (использовать экранные компоненты);
- Осуществить визуализацию графических примитивов

Важное замечание: должна быть предусмотрена возможность приостановки выполнения визуализации, изменения параметров «на лету» и снятия скриншотов с сохранением в текущую активную директорию.

Для всех динамических сцен необходимо задавать параметр скорости!

1) Создать класс Triangle и класс Point. Объявить список из n объектов класса Point, написать функцию, определяющую, какая из точек лежит внутри, а какая – снаружи треугольника.

```
from datetime import datetime
import pygame
```

```
pygame.init()
```

```
WIDTH = 800
HEIGHT = 600
SCREEN = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Движение окружности")
```

```
class Circle:
    def __init__(self, x_pos, y_pos, radius, speed_x, speed_y):
        self.x = x_pos
        self.y = y_pos
        self.r = radius
        self.sx = speed_x
        self.sy = speed_y

    def move(self):
        self.x += self.sx
        self.y += self.sy
```

```
if self.x - self.r <= 0 or self.x + self.r >= WIDTH:
    self.sx = -self.sx
if self.y - self.r <= 0 or self.y + self.r >= HEIGHT:
    self.sy = -self.sy
```

```
def draw(self):
    pygame.draw.circle(SCREEN, (255, 0, 0), (int(self.x), int(self.y)), self.r)
```

```
CIRCLE = Circle(WIDTH // 2, HEIGHT // 2, 30, 5, 3)
RUNNING = True
PAUSED = False
CLOCK = pygame.time.Clock()
```

```
while RUNNING:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            RUNNING = False
        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_SPACE:
                PAUSED = not PAUSED
            if event.key == pygame.K_s:
                SCREENSHOT_NAME = f"screenshot_{datetime.now().strftime('%H%M%S')}.png"
                pygame.image.save(SCREEN, SCREENSHOT_NAME)
                print(f"Сохранен: {SCREENSHOT_NAME}")
```

```
SCREEN.fill((0, 0, 0))
```

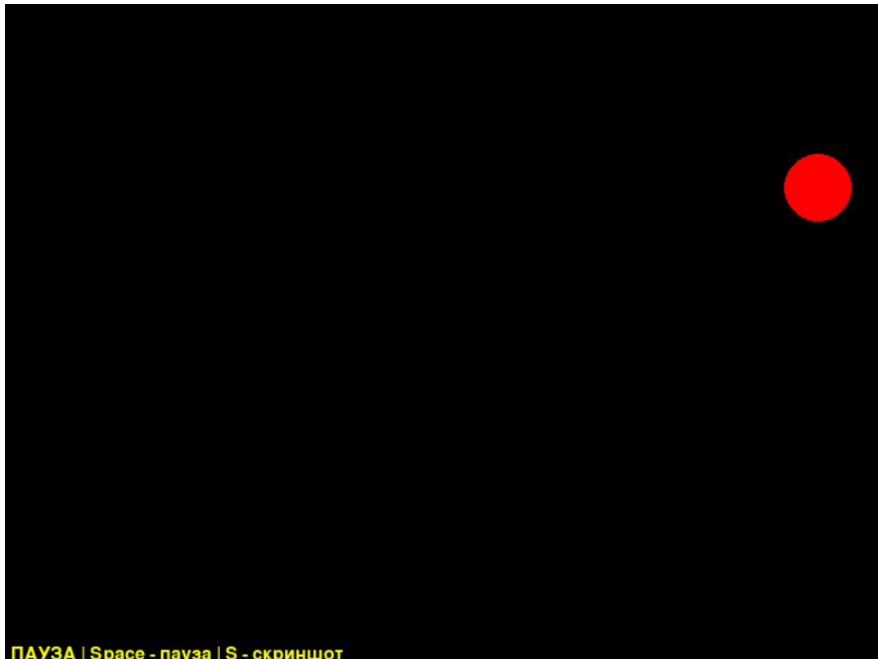
```
if not PAUSED:
    CIRCLE.move()
```

```
CIRCLE.draw()
```

```
FONT = pygame.font.Font(None, 24)
STATUS_TEXT = "ПАУЗА" if PAUSED else "ДВИЖЕНИЕ"
TEXT_COLOR = (255, 255, 0) if PAUSED else (0, 255, 0)
TEXT_SURFACE = FONT.render(f"{STATUS_TEXT} | Space - пауза | S - скриншот", True,
TEXT_COLOR)
SCREEN.blit(TEXT_SURFACE, (10, HEIGHT - 30))
```

```
pygame.display.flip()
CLOCK.tick(60)
```

```
pygame.quit()
```



Задание 2. Реализовать построение заданного типа фрактала по варианту Везде, где это необходимо, предусмотреть ввод параметров, влияющих на внешний вид фрактала

1) Множество Мальдеброта

```
import math
from datetime import datetime
import pygame
```

```
pygame.init()
```

```
WIDTH = 800
HEIGHT = 600
SCREEN = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Дерево Пифагора")
CLOCK = pygame.time.Clock()
```

```
def draw_tree(x, y, length, angle, tree_depth, wind_power):
    if tree_depth == 0 or length < 2:
        return
```

```
    wind_effect = wind_power * tree_depth
```

```
x2 = x + length * math.cos(math.radians(angle + wind_effect))
y2 = y - length * math.sin(math.radians(angle + wind_effect))
```

```
red = min(139, 80 + tree_depth * 10)
green = min(69, 40 + tree_depth * 5)
blue = min(19, 10 + tree_depth)
color = (red, green, blue)
```

```
if tree_depth < 4:
    color = (34, 139, 34)
```

```
pygame.draw.line(SCREEN, color, (int(x), int(y)), (int(x2), int(y2)), max(1, tree_depth // 2))
```

```
new_len = length * 0.7
draw_tree(x2, y2, new_len, angle - 35 - wind_power, tree_depth - 1, wind_power)
draw_tree(x2, y2, new_len, angle + 35 - wind_power, tree_depth - 1, wind_power)
```

```
DEPTH = 10
WIND = 0
RUNNING = True
```

```
while RUNNING:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            RUNNING = False
        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_UP:
                DEPTH = min(DEPTH + 1, 12)
            if event.key == pygame.K_DOWN:
                DEPTH = max(DEPTH - 1, 1)
            if event.key == pygame.K_LEFT:
                WIND -= 3
            if event.key == pygame.K_RIGHT:
                WIND += 3
            if event.key == pygame.K_SPACE:
                WIND = 0
            if event.key == pygame.K_s:
                SCREENSHOT_NAME = f"tree_{datetime.now().strftime('%H%M%S')}.png"
                pygame.image.save(SCREEN, SCREENSHOT_NAME)
                print(f"Сохранен: {SCREENSHOT_NAME}")
```

```
SCREEN.fill((0, 0, 0))
draw_tree(WIDTH // 2, HEIGHT - 100, 100, -90, DEPTH, WIND)
```

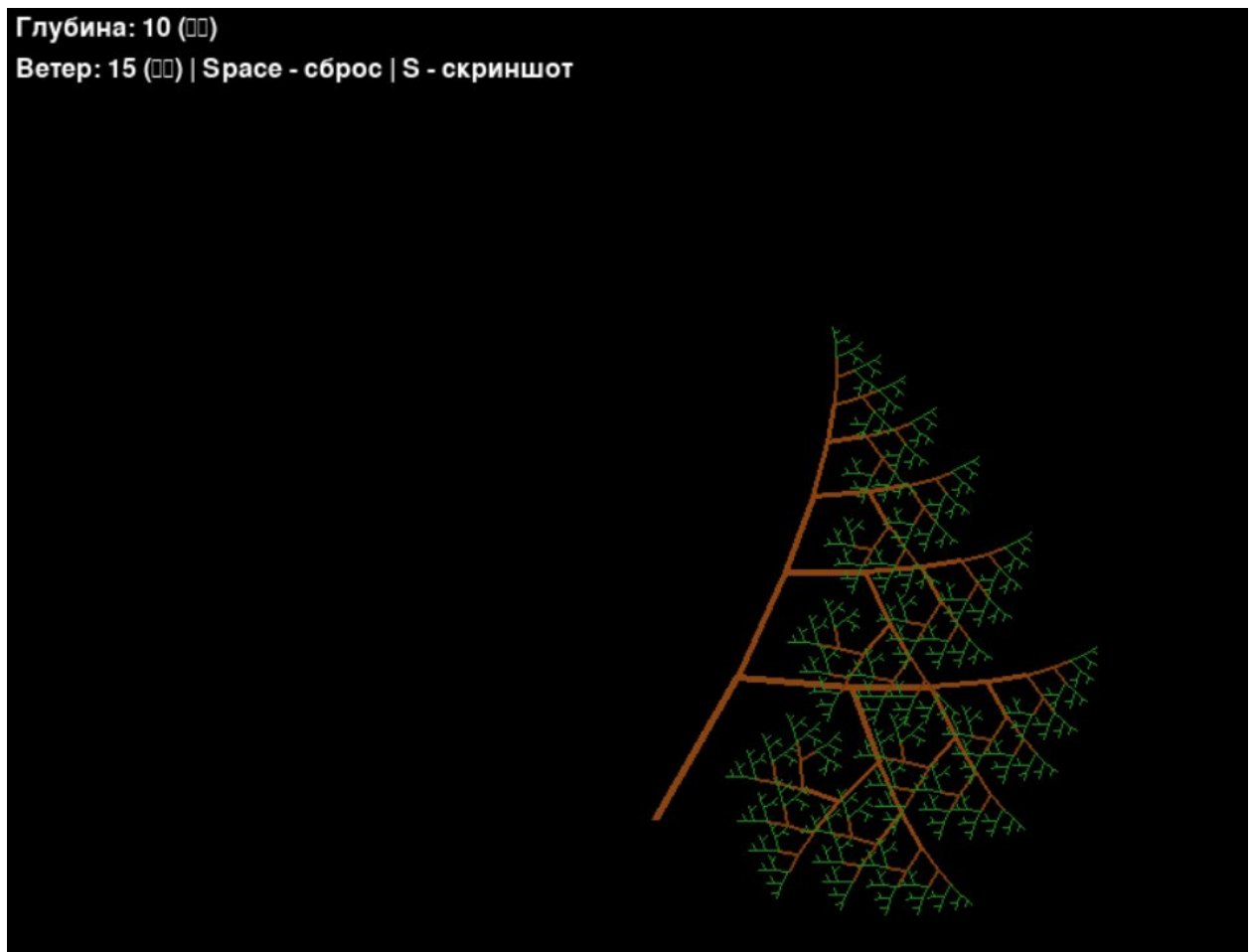
```

FONT = pygame.font.Font(None, 24)
TEXT1 = FONT.render(f"Глубина: {DEPTH} (↑ ↓)", True, (255, 255, 255))
TEXT2 = FONT.render(f"Ветер: {WIND} (←→) | Space - сброс | S - скриншот", True, (255, 255, 255))
SCREEN.blit(TEXT1, (10, 10))
SCREEN.blit(TEXT2, (10, 35))

pygame.display.flip()
CLOCK.tick(60)

pygame.quit()

```



Вывод работы: освоил возможности языка программирования Python в разработке оконных приложений